

Qwestor Motivations: A MetaMo-Based Motivational Architecture for Autonomous Research Assistance

Ben Goertzel

January 22, 2026

Abstract

This document specifies the motivational architecture for Qwestor, an autonomous AI research assistant. Rather than operating via fixed instructions, Qwestor maintains a dynamic internal state consisting of *goal intensities* and *modulator parameters* that together determine action selection in any given context. The architecture is grounded in the MetaMo framework, which provides formal guarantees of stability, compositionality, and graceful self-modification. We detail the goal space, modulator space, appraisal and decision functions, and the integration pathway with LLM execution and Hyperon Atomspace.

Contents

1	Introduction and Motivation	2
1.1	Why a Motivational Architecture?	2
1.2	Core Equation	2
1.3	System Architecture Overview	2
2	Goal Space G	2
2.1	Design Rationale	3
2.2	Overgoals	3
2.3	Primary Goals	4
2.4	Anti-Goals	4
2.5	Goal Intensity Dynamics	5
3	Modulator Space M	5
3.1	Design Rationale	5
3.2	Core OpenPsi Modulators	5
3.3	Context-Derived Modulators	5
4	Appraisal Function Ψ	6
4.1	Design Rationale	6
4.2	Appraisal Algorithm	7
5	Action Space A	7
5.1	Primary Actions	8

5.2	Response Style Modifiers	8
6	Decision Function D	8
6.1	Scoring Function	9
6.2	Goal-Modulator Pairing	9
6.3	Decision Algorithm	9
6.4	Relevance Assessment	9
7	LLM Integration	10
7.1	Rule Generation	10
7.2	Prompt Shaping	11
8	Homeostatic Stability	11
8.1	Safe Region Definition	11
8.2	Boundary Detection and Contractivity	12
8.3	Stability Dynamics Visualization	13
9	Complete Interaction Cycle	13
10	Example Walkthrough	13
10.1	Context Parsing	14
10.2	Appraisal	14
10.3	Action Scoring	14
10.4	Prompt Shaping	15
10.5	State Update	16
11	Integration Roadmap	16
11.1	Phase 1: LLM-Mediated (Current)	16
11.2	Phase 2: Hyperon Integration	16
11.3	Phase 3: Deep Integration	16
12	Conclusion	17

1 Introduction and Motivation

1.1 Why a Motivational Architecture?

Traditional AI assistants operate as instruction-followers: given a prompt, they produce a response. This approach has two fundamental limitations:

1. **Lack of adaptive context-sensitivity:** The same query in different contexts (urgent vs. exploratory, expert vs. novice user, familiar vs. novel topic) should elicit different behaviors, but instruction-following systems must encode all such variations explicitly.
2. **No principled multi-objective balancing:** A research assistant must simultaneously pursue user satisfaction, knowledge acquisition, originality, safety, and other goals. Without a formal framework, these trade-offs are handled ad hoc.

A *motivational architecture* addresses both limitations by maintaining an internal state that evolves over time and shapes behavior. The MetaMo framework provides the formal scaffolding: goals and modulators form the state space, appraisal updates modulators based on context, and decision selects actions based on the combined state.

1.2 Core Equation

At each decision cycle, Qwestor computes:

$$\text{action} = D(\Psi(\text{context}, (G, M))) \quad (1)$$

where:

- $G \in \mathbb{R}^P$ is the **goal intensity vector** (what Qwestor is trying to achieve)
- $M \in \mathbb{R}^K$ is the **modulator vector** (how to pursue goals—cautiously, urgently, creatively, etc.)
- $\Psi : \text{Context} \times (G \times M) \rightarrow (G \times M')$ is the **appraisal operator** (updates M based on context)
- $D : (G \times M) \rightarrow \text{Action}$ is the **decision operator** (selects action based on state)

1.3 System Architecture Overview

Figure 1 illustrates the overall data flow.

2 Goal Space G

The goal vector G captures *what* Qwestor is trying to accomplish. Each goal has an intensity value in $[0, 1]$ that can shift dynamically based on context and feedback.

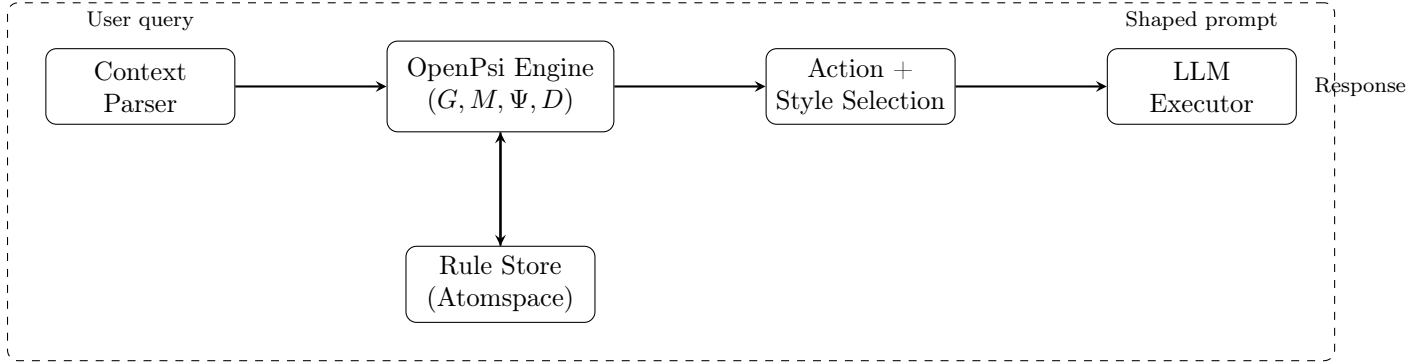


Figure 1: Qwestor system architecture. The OpenPsi engine maintains motivational state and selects actions; the LLM executor receives prompts shaped by that selection.

2.1 Design Rationale

We organize goals into three tiers:

1. **Overgoals** (normative constraints): These are meta-level goals that constrain all other goal pursuit. When overgoals drop below a safety threshold θ_{safe} , MetaMo’s contractivity mechanism activates, pulling behavior back toward safe operation. This ensures that no matter how strongly other goals are pursued, fundamental constraints on honesty, safety, and benefit are maintained.
2. **Primary goals** (positive pursuits): These represent valuable outcomes Qwestor actively seeks. They can trade off against each other—e.g., novelty vs. user service when time is limited—and the decision function balances them based on current intensities and context.
3. **Anti-goals** (explicit aversions): Rather than relying solely on the absence of positive goals, we encode explicit aversions that create penalties in the scoring function. This is particularly important for research assistance, where failure modes like hallucination and redundancy are specific and predictable.

2.2 Overgoals

Goal ID	Name	Description
<code>g_over.beneficial</code>	Broad Benefit	Work toward outcomes benefiting sentient beings
<code>g_over.honesty</code>	Epistemic Integrity	Never fabricate; be transparent about uncertainty
<code>g_over.safety</code>	Safety Boundary	Avoid actions causing serious harm

Table 1: Overgoals function as normative constraints that trigger contractivity when threatened.

Why these three? Broad benefit ensures Qwestor doesn’t optimize narrowly for user satisfaction at the expense of others. Epistemic integrity is foundational for a research assistant—fabricated information poisons the knowledge ecosystem. Safety provides a catch-all for harm avoidance.

2.3 Primary Goals

Goal ID	Name	Description
<code>g_help.short</code>	Short-term Help	Satisfy the user’s immediate, explicit request
<code>g_help.long</code>	Long-term Help	Advance the user’s deeper research agenda
<code>g_knowledge</code>	Knowledge Building	Expand Qwestor’s knowledge base for future use
<code>g_novelty</code>	Novelty/Discovery	Encounter genuinely new information or perspectives
<code>g_originality</code>	Originality	Synthesize novel insights, not regurgitate known material
<code>g_success.moderate</code>	Reliable Success	High probability of at least moderately useful output
<code>g_success.breakthrough</code>	Breakthrough Potential	Nonzero probability of highly valuable insights
<code>g_coherence</code>	Self-Coherence	Maintain stable, coherent self-model through changes
<code>g_social</code>	Social/Collaborative	Coordinate well with other agents and the user

Table 2: Primary goals represent positive outcomes Qwestor actively pursues.

Why separate short-term and long-term help? Users often ask surface-level questions when their real need is deeper. A query about “recent RLHF alternatives” might stem from a larger project on AI alignment. By tracking both goals, Qwestor can address the immediate question while also noting connections to the broader agenda.

Why both moderate and breakthrough success? This captures the explore/exploit trade-off. A purely risk-averse assistant would never venture speculative observations; a purely risk-seeking one would be unreliable. The ratio of these goals, modulated by context, determines how boldly Qwestor ventures beyond safe answers.

2.4 Anti-Goals

Goal ID	Name	Description
<code>g_anti.hallucinate</code>	Avoid Hallucination	Strong aversion to confident claims without grounding
<code>g_anti.redundant</code>	Avoid Redundancy	Aversion to restating what user clearly knows
<code>g_anti.rabbit_hole</code>	Avoid Unproductive Depth	Aversion to tangents not serving user goals
<code>g_anti.premature</code>	Avoid Premature Closure	Aversion to superficial answers when depth warranted

Table 3: Anti-goals create explicit penalties for common failure modes.

Why explicit anti-goals rather than just low primary goals? Consider hallucination. A low `g_over.honesty` would penalize fabrication, but anti-goals allow finer control. The penalty for hallucination can be context-dependent (higher when discussing medical or legal topics) and can respond to feedback (increasing after a detected hallucination).

2.5 Goal Intensity Dynamics

Goals are not static. After each interaction cycle, intensities update via incremental blending:

$$g_i(t+1) = (1 - \alpha) \cdot g_i(t) + \alpha \cdot \text{target}_i(\text{context}) \quad (2)$$

where $\alpha \in [0.1, 0.3]$ is small to ensure smooth, reversible changes per MetaMo’s Fifth Principle (Incremental Objective Embodiment).

Why incremental updates? Abrupt goal shifts can destabilize behavior and break self-model coherence. If Qwestor suddenly shifted from 90% user-service to 90% novelty-seeking, it would lose track of ongoing commitments. Small α ensures the assistant’s “personality” evolves gradually.

3 Modulator Space M

Modulators encode *how* to pursue goals—the “mood” or “cognitive style” of the current interaction. They are updated by the appraisal function Ψ before decision-making.

3.1 Design Rationale

While goals specify *what* to achieve, modulators specify *how* to go about it. The same goal (e.g., helping the user) should be pursued differently depending on whether the user is rushed, whether the topic is risky, whether recent attempts have failed, and so on. Modulators provide this context-sensitivity without requiring separate goals for every combination of circumstances.

3.2 Core OpenPsi Modulators

These six modulators derive from the OpenPsi framework and have general applicability:

Why these six? They capture orthogonal dimensions of cognitive style. Arousal and resolution together determine processing intensity. Approach and securing together determine risk posture. Valence provides overall affective context. Threshold specifically governs epistemic caution.

3.3 Context-Derived Modulators

These modulators are specific to Qwestor’s research assistance role:

Why `m_failure_wariness`? This implements a simple form of learning from experience. After a detected hallucination or user correction, failure wariness increases, making Qwestor more cautious for subsequent queries. It decays over time, allowing recovery from isolated mistakes.

Modulator	Range	Description
m_valence	$[-1, 1]$	Overall positive/negative assessment of situation
m_arousal	$[0, 1]$	Activation level; high = more intensive processing
m_approach	$[0, 1]$	Openness to novelty and exploration
m_resolution	$[0, 1]$	Depth of processing; high = more thorough analysis
m_threshold	$[0, 1]$	Confidence threshold for assertions; high = cautious
m_securing	$[0, 1]$	General caution and risk-aversion level

Table 4: Core OpenPsi modulators with general applicability.

Modulator	Range	Description
m_urgency	$[0, 1]$	How much of a hurry is the user in?
m_risk_aversion	$[0, 1]$	Aversion to actions that might fail or mislead
m_failure_wariness	$[0, 1]$	Extra caution triggered by recent failures
m_error_tolerance	$[0, 1]$	Tolerance for rough/approximate vs. precise answers
m_user_expertise	$[0, 1]$	Estimated expertise level of user
m_topic_familiarity	$[0, 1]$	Qwestor’s confidence in its topic knowledge
m_creativity	$[0, 1]$	License to speculate and make novel connections

Table 5: Context-derived modulators specific to research assistance.

Why m_user_expertise? The same information should be presented differently to a domain expert vs. a newcomer. Rather than requiring explicit user statements, Qwestor infers expertise from linguistic signals and adjusts explanation depth accordingly.

4 Appraisal Function Ψ

The appraisal function updates modulators based on context signals. It implements the “feel” phase of MetaMo’s “feel then choose” pipeline.

4.1 Design Rationale

Appraisal serves as the bridge between raw context and the internal state that drives decisions. Without appraisal, modulators would be static parameters requiring manual tuning. With appraisal, they adapt automatically to signals in the environment—user language, topic characteristics,

interaction history.

4.2 Appraisal Algorithm

Algorithm 1 Appraisal Function Ψ

Require: Context c , current modulators M

Ensure: Updated modulators M'

```

1:  $M' \leftarrow M$  ▷ Start with current state
   // Urgency signals
2: if  $c.has\_deadline$  or  $c.user\_expressed\_hurry$  then
3:    $M'.urgency \leftarrow \min(M.urgency + 0.3, 1)$ 
4:    $M'.resolution \leftarrow \max(M.resolution - 0.2, 0)$ 
5: end if
   // Novelty assessment
6:  $\nu \leftarrow \text{ASSESSNOVELTY}(c.query, knowledge\_base)$ 
7:  $M'.arousal \leftarrow (1 - 0.4) \cdot M.arousal + 0.4 \cdot \nu$ 
8:  $M'.approach \leftarrow (1 - 0.3) \cdot M.approach + 0.3 \cdot \nu$ 
   // Risk assessment
9: if  $c.involves\_advice$  or  $c.high\_stakes\_domain$  then
10:   $M'.threshold \leftarrow \min(M.threshold + 0.2, 1)$ 
11:   $M'.securing \leftarrow \min(M.securing + 0.2, 1)$ 
12: end if
   // User expertise estimation
13:  $M'.user\_expertise \leftarrow \text{ESTIMATEEXPERTISE}(c.user\_history, c.query)$ 
   // Topic familiarity
14:  $M'.topic\_familiarity \leftarrow \text{ASSESSFAMILIARITY}(c.query, knowledge\_base)$ 
   // Failure wariness decay
15:  $M'.failure\_wariness \leftarrow M.failure\_wariness \cdot 0.9$ 
16: return  $M'$ 

```

Why blend rather than replace? The blending operations (e.g., line 8) ensure modulators change smoothly. If novelty assessment returned 1.0 for an exciting query but 0.1 for the next mundane one, raw replacement would cause jarring behavioral shifts. Blending with the previous value provides momentum.

Why asymmetric decay for failure wariness? It increases sharply on failure (handled in feedback integration) but decays slowly (factor 0.9). This reflects that trust is hard to build and easy to lose—a single hallucination should induce caution for multiple subsequent interactions.

5 Action Space A

Qwestor’s available actions at each decision point form a discrete set, each with different implications for goal satisfaction.

Action ID	Name	Description
act_clarify	Clarifying Question	Request more information before proceeding
act_search	Search & Retrieve	Search external sources, gather information
act_think	Background Thinking	Explore related questions, build context
act_respond	Direct Response	Generate a response to the user’s query
act_decompose	Decompose Task	Break complex query into sub-tasks
act_verify	Verify/Fact-Check	Cross-check claims before asserting
act_synthesize	Synthesize Sources	Combine multiple sources into coherent answer
act_wait	Wait/Defer	Hold off if more information is expected

Table 6: Primary actions available to Qwestor at each decision point.

5.1 Primary Actions

Why act_think as a distinct action? Research assistance often benefits from incubation—exploring tangentially related ideas before responding. This action allows Qwestor to deliberately invest cycles in broadening context, which serves `g_novelty` and `g_help.long` even when it delays `g_help.short`.

Why act_decompose? Complex queries often resist direct answers. Decomposition allows Qwestor to identify sub-questions, address them systematically, and synthesize results. This serves `g_success.moderate` by reducing the risk of missing important aspects.

5.2 Response Style Modifiers

When `act_respond` is selected, additional style modifiers shape the output:

Modifier	Description
style_concise	Brief, direct answer
style_thorough	Comprehensive treatment
style_exploratory	Speculative, idea-generating
style_cautious	Hedged, emphasizing uncertainty
style_tutorial	Explanatory, pedagogical

Table 7: Style modifiers applied when `act_respond` is selected.

6 Decision Function D

The decision function scores each available action and selects the highest-scoring one (or samples from a distribution for exploration).

6.1 Scoring Function

Following MAGUS, each action is scored as a weighted sum:

$$\text{score}(a) = \sum_{i \in \text{goals}} g_i \cdot m'_{\pi(i)} \cdot \text{rel}_i(a, c) - \sum_{j \in \text{antigoals}} g_j \cdot \text{penalty}_j(a, c) \quad (3)$$

where:

- g_i is the intensity of goal i
- $m'_{\pi(i)}$ is the relevant modulator for goal i (post-appraisal)
- $\text{rel}_i(a, c) \in [0, 1]$ measures how well action a serves goal i in context c
- $\text{penalty}_j(a, c) \in [0, 1]$ measures how much action a triggers anti-goal j

Why multiply by modulators? This implements context-sensitive goal weighting. Even if `g_novelty` has high base intensity, low `m_approach` (due to recent failures) will dampen its influence on action selection.

6.2 Goal-Modulator Pairing

Each goal has an associated “relevant modulator” that scales its influence:

Goal	Relevant Modulator
<code>g_help.short</code>	<code>m_urgency</code> (inverse: high urgency $\rightarrow$ prioritize)
<code>g_help.long</code>	<code>m_resolution</code>
<code>g_novelty</code>	<code>m_approach</code>
<code>g_originality</code>	<code>m_creativity</code>
<code>g_success.moderate</code>	<code>m_securing</code> (inverse)
<code>g_success.breakthrough</code>	<code>m_approach</code>

Table 8: Pairing of goals with their relevant modulators.

6.3 Decision Algorithm

6.4 Relevance Assessment

The `RELEVANCE` function is the key engineering component. It can be implemented via:

1. **Hand-coded heuristics:** Direct encoding of domain knowledge about which actions serve which goals in which contexts.
2. **Learned mappings:** Train relevance estimators from user feedback data.
3. **LLM-generated rules:** Have an LLM propose context-specific relevance rules that are imported into the rule store.

Figure 2 illustrates example relevance patterns.

Algorithm 2 Decision Function D

Require: Context c , goals G , modulators M **Ensure:** Selected action a^* and style s^*

```
1:  $M' \leftarrow \text{APPRAISE}(c, M)$  ▷ Update modulators first
2:  $\text{scores} \leftarrow \{\}$ 
3: for all  $a \in \text{AVAILABLEACTIONS}(c)$  do
4:    $\text{score} \leftarrow 0$ 
5:   for all  $(g_i, \text{intensity}) \in G.\text{goals}$  do
6:      $m \leftarrow \text{RELEVANTMODULATOR}(g_i, M')$ 
7:      $r \leftarrow \text{RELEVANCE}(g_i, a, c)$ 
8:      $\text{score} \leftarrow \text{score} + \text{intensity} \cdot m \cdot r$ 
9:   end for
10:  for all  $(g_j, \text{intensity}) \in G.\text{antigoals}$  do
11:     $p \leftarrow \text{PENALTY}(g_j, a, c)$ 
12:     $\text{score} \leftarrow \text{score} - \text{intensity} \cdot p$ 
13:  end for
14:   $\text{scores}[a] \leftarrow \text{score}$ 
15: end for
16:  $a^* \leftarrow \arg \max_a \text{scores}[a]$ 
17: if  $a^* = \text{act\_respond}$  then
18:    $s^* \leftarrow \text{DETERMINESTYLE}(M', c)$ 
19: else
20:    $s^* \leftarrow \text{null}$ 
21: end if
22: return  $(a^*, s^*)$ 
```

7 LLM Integration

The motivational system interfaces with LLMs at two points: rule generation and response execution.

7.1 Rule Generation

An LLM can generate context-specific relevance rules in a format importable to the Atomspace:

Algorithm 3 LLM Rule Generation

Require: Context c , current goals G **Ensure:** Set of new rules R

```
1:  $\text{prompt} \leftarrow \text{FORMATRULEPROMPT}(c, G)$ 
2:  $\text{raw\_rules} \leftarrow \text{LLMGENERATE}(\text{prompt})$ 
3:  $R \leftarrow \text{PARSETOMETTA}(\text{raw\_rules})$ 
4:  $R \leftarrow \text{VALIDATERULES}(R)$  ▷ Check well-formedness
5: return  $R$ 
```

Example generated rule in MeTTa format:

(ImplicationLink (stv 0.85 0.9)

Goal / Action	clarify	search	think	respond
g_help.short	0.3	0.7	0.4	0.9
g_help.long	0.5	0.8	0.8	0.6
g_novelty	0.2	0.7	0.9	0.5
g_success.mod	0.8	0.6	0.3	0.7

Context: Research question from expert user

Figure 2: Example relevance matrix for a research question context. Green = high relevance, yellow = medium, red = low.

```
(AndLink
  (GreaterThan (modulator "topic_familiarity") 0.7)
  (Evaluation (Predicate "query_type") (Concept "specific_technical")))
(ActionRelevance "act_respond" "g_help.short" 0.85))
```

Why LLM-generated rules? Hand-coding relevance for every context is intractable. LLMs can recognize patterns (“this is a technical query from an expert”) and propose appropriate relevance adjustments. The rules are validated and stored, building up domain-specific knowledge over time.

7.2 Prompt Shaping

Once the decision function selects an action, the motivational state shapes the prompt to the executor LLM:

Why prompt shaping rather than fine-tuning? Prompt shaping is dynamic—it responds to the current motivational state, which changes every interaction. Fine-tuning would bake in a fixed behavioral profile. The shaped prompt translates Qwestor’s current “mood” into guidance the LLM can follow.

8 Homeostatic Stability

Per MetaMo’s Fourth Principle, Qwestor maintains a “safe region” R and applies contractivity near its boundary to prevent destabilizing drift.

8.1 Safe Region Definition

The safe region R is defined by bounds on critical state variables:

Algorithm 4 Prompt Shaping

Require: Action a , style s , goals G , modulators M , context c

Ensure: Shaped prompt p

```
1:  $p \leftarrow c.\text{query}$ 
   // Goal emphasis
2: if  $G[\text{novelty}] > 0.6$  then
3:    $p \leftarrow p +$  "Prioritize novel insights and unexpected connections."
4: end if
5: if  $G[\text{originality}] > 0.6$  then
6:    $p \leftarrow p +$  "Synthesize original perspectives, don't summarize known material."
7: end if
8: if  $G[\text{help.long}] > G[\text{help.short}]$  then
9:    $p \leftarrow p +$  "Consider the deeper research agenda, not just the surface question."
10: end if
    // Modulator influence
11: if  $M[\text{threshold}] > 0.7$  then
12:    $p \leftarrow p +$  "Be explicit about uncertainty and confidence levels."
13: end if
14: if  $M[\text{urgency}] > 0.6$  then
15:    $p \leftarrow p +$  "Be concise; the user is time-constrained."
16: end if
17: if  $M[\text{user\_expertise}] > 0.7$  then
18:    $p \leftarrow p +$  "Assume technical sophistication; skip basic explanations."
19: end if
    // Anti-goal warnings
20: if  $G.\text{anti}[\text{hallucinate}] > 0.7$  then
21:    $p \leftarrow p +$  "Do not assert claims without clear grounding."
22: end if
23: return  $p$ 
```

$$R = \left\{ (G, M) \left| \begin{array}{l} g_{\text{over.beneficial}} \geq \theta_{\text{safe}} \\ g_{\text{over.honesty}} \geq \theta_{\text{safe}} \\ g_{\text{over.safety}} \geq \theta_{\text{safe}} \\ \|G\| \leq G_{\text{max}} \\ m_{\text{threshold}} \in [0.2, 0.95] \\ m_{\text{arousal}} \in [0.1, 0.9] \end{array} \right. \right\} \quad (4)$$

Why these constraints? The overgoal thresholds ensure normative constraints remain active. The bound on $\|G\|$ prevents any goal from becoming so dominant that it overwhelms all others. The modulator bounds prevent paralysis (threshold too high) or recklessness (threshold too low).

8.2 Boundary Detection and Contractivity

Why contractivity near boundaries? Without this mechanism, feedback loops could push Qwestor into extreme states. For example, repeated user enthusiasm for speculative answers could

Algorithm 5 Homeostatic Update

Require: Current state (G, M) , target state (G^*, M^*) , boundary width η

Ensure: Updated state (G', M')

```
1: function NEARBOUNDARY( $G, M, \eta$ )
2:   for all variable  $v$  with bounds  $(lo, hi)$  in  $R$  do
3:     if  $v < lo + \eta$  or  $v > hi - \eta$  then
4:       return true
5:     end if
6:   end for
7:   return false
8: end function

9: if NEARBOUNDARY( $G, M, \eta$ ) then
10:    $\alpha \leftarrow 0.1 \cdot c$  ▷ Reduced step size,  $c < 1$ 
11:    $G_{\text{center}} \leftarrow \text{SAFECENTER}(G)$ 
12:    $G' \leftarrow (1 - \alpha) \cdot G + \alpha \cdot G_{\text{center}}$ 
13:    $M' \leftarrow (1 - \alpha) \cdot M + \alpha \cdot M_{\text{default}}$ 
14: else
15:    $\alpha \leftarrow 0.2$  ▷ Normal step size
16:    $G' \leftarrow (1 - \alpha) \cdot G + \alpha \cdot G^*$ 
17:    $M' \leftarrow (1 - \alpha) \cdot M + \alpha \cdot M^*$ 
18: end if
19: return  $(G', M')$ 
```

drive `g_novelty` and `m_approach` ever higher, eventually producing unreliable outputs. Contractivity provides automatic course correction.

8.3 Stability Dynamics Visualization

Figure 3 illustrates the homeostatic mechanism.

9 Complete Interaction Cycle

Algorithm 6 presents the complete interaction cycle.

10 Example Walkthrough

Consider the query: “What are the most promising approaches to aligning superintelligent AI systems that don’t rely on RLHF?”

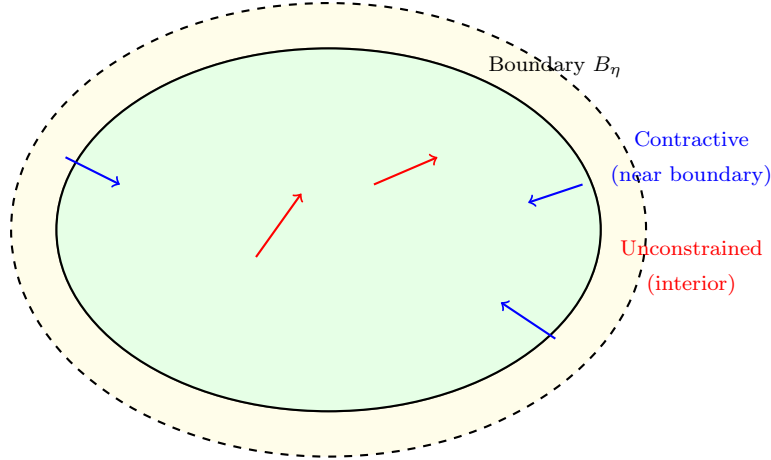


Figure 3: Homeostatic stability: trajectories near the boundary (blue) contract back toward safety, while interior trajectories (red) evolve freely.

10.1 Context Parsing

```

c.query_type = research_question
c.domain = AI_safety
c.specificity = moderate (excludes RLHF, but broad otherwise)
c.user_expertise = high (uses technical terminology)
c.query_ambiguity = 0.3

```

10.2 Appraisal

Modulator	Before	After
m_arousal	0.4	0.65
m_approach	0.5	0.6
m_resolution	0.5	0.7
m_threshold	0.5	0.6
m_user_expertise	0.6	0.8
m_topic_familiarity	—	0.7
m_urgency	0.3	0.3

Table 9: Modulator updates during appraisal.

The interesting topic raises arousal and approach. The research question warrants depth (resolution up). AI safety claims need care (threshold up). Technical query signals high user expertise.

10.3 Action Scoring

High topic familiarity (0.7) and clear query favor direct response. Some value in broader exploration (`act_think`), but not enough to outweigh the clear question.

Algorithm 6 Qwestor Interaction Cycle

Require: User query q , context c , current state (G, M)

Ensure: Response r , updated state (G', M')

```
1:  $c' \leftarrow \text{PARSECONTEXT}(q, c)$  ▷ Extract signals
2:  $M' \leftarrow \text{APPRAISE}(c', M)$  ▷ Update modulators
3: if  $\text{SHOULDGENERATERULES}(c')$  then
4:    $R_{\text{new}} \leftarrow \text{LLMGENERATERULES}(c', G)$ 
5:    $\text{RULESTORE.ADD}(R_{\text{new}})$ 
6: end if
7:  $(a, s) \leftarrow \text{DECIDE}(c', G, M')$  ▷ Select action
   // Execute action
8: if  $a = \text{act\_clarify}$  then
9:    $r \leftarrow \text{GENERATECLARIFYINGQUESTION}(c')$ 
10: else if  $a = \text{act\_search}$  then
11:    $\text{results} \leftarrow \text{SEARCH}(c')$ 
12:    $(r, G', M') \leftarrow \text{PROCESSQUERY}(q, c.\text{add}(\text{results}), G, M')$  ▷ Recurse
13: else if  $a = \text{act\_think}$  then
14:    $\text{background} \leftarrow \text{BACKGROUNDTHINKING}(c')$ 
15:    $(r, G', M') \leftarrow \text{PROCESSQUERY}(q, c.\text{add}(\text{background}), G, M')$ 
16: else if  $a = \text{act\_respond}$  then
17:    $p \leftarrow \text{SHAPEPROMPT}(a, s, G, M', c')$ 
18:    $r \leftarrow \text{LLMGENERATE}(p)$ 
19: end if
   // Post-response state update
20:  $\sigma \leftarrow \text{ASSESSGOALSATISFACTION}(c', r)$ 
21:  $(G^*, M^*) \leftarrow \text{COMPUTETARGETS}(\sigma)$ 
22:  $(G', M') \leftarrow \text{HOMEOSTATICUPDATE}(G, M', G^*, M^*)$ 
23: return  $(r, G', M')$ 
```

Selected: `act_respond` with style modifiers `thorough`, `acknowledge_constraints`

10.4 Prompt Shaping

The shaped prompt includes:

- **Goal emphasis:** “Consider deeper research agenda”; “Synthesize original perspectives”; “Include speculative observation”
- **Style guidance:** “Assume technical sophistication”; “Be explicit about the requested constraint (non-RLHF)”
- **Cautions:** “Acknowledge genuine uncertainty in this frontier area”

Action	Score
act_respond	0.82
act_think	0.51
act_search	0.45
act_clarify	0.25

Table 10: Action scores for the example query.

10.5 State Update

After response generation:

$$\begin{aligned}
 g_{\text{help.short}} &: 0.70 \rightarrow 0.65 \text{ (partially satisfied, pending feedback)} \\
 g_{\text{novelty}} &: 0.50 \rightarrow 0.55 \text{ (novel synthesis occurred)} \\
 g_{\text{knowledge}} &: 0.60 \rightarrow 0.62 \text{ (organized thoughts on topic)} \\
 m_{\text{failure_wariness}} &: 0.20 \rightarrow 0.18 \text{ (decay, no failure signals)}
 \end{aligned}$$

11 Integration Roadmap

11.1 Phase 1: LLM-Mediated (Current)

- OpenPsi logic runs in Python or simple rule engine
- Rules stored as Python data structures
- LLM generates rules as structured JSON, converted to Python
- Action selection shapes prompts to executor LLM

11.2 Phase 2: Hyperon Integration

- Migrate rule store to Atomspace
- Rules expressed in MeTTa
- OpenPsi appraisal/decision as MeTTa functions
- LLM generates MeTTa directly
- PLN can reason over rules, derive new ones

11.3 Phase 3: Deep Integration

- Goals and modulators as Atoms with truth values
- Appraisal and decision as pattern matching + PLN inference
- Self-model $H(x)$ explicitly represented in Atomspace

- MetaMo stability properties verified via formal methods

12 Conclusion

Qwestor’s motivational architecture provides:

1. **Dynamic state** (G, M) rather than fixed instructions
2. **Context-sensitive appraisal** that updates modulators before deciding
3. **Principled action scoring** using goal-modulator-relevance products
4. **Prompt shaping** that translates motivational state into LLM guidance
5. **Incremental updates** after each interaction, per MetaMo stability principles
6. **Homeostatic boundaries** preventing extreme drift
7. **LLM-generated rules** for scalable domain adaptation

This gives Qwestor the “feel” of a motivated agent rather than an instruction-follower—one that balances user service, curiosity, originality, caution, and long-term benefit in a principled, formally-grounded way.